

Feature Engineering & Preprocessing

Topics Covered

Scaling & Normalization
Encoding Categorical Variables
Handling Skewness
Feature Selection Methods

Table of Contents

1. Scaling & Normalization	3
2. Encoding Categorical Variables	6
3. Handling Skewness.....	9
4. Feature Selection Methods	12
5. Quick Reference Cheat Sheet.....	15

1. Scaling & Normalization

1.1 Why It Matters

Many ML algorithms are sensitive to the scale of input features. Distance-based models (KNN, K-Means, SVM), gradient-descent-based models (linear/logistic regression, neural networks), and regularized models (Ridge, Lasso) all assume features live on comparable scales. Without scaling, a feature like "income" (0–1,000,000) can dominate a feature like "age" (0–100), even if age is more predictive.

Tip: Tree-based models (Decision Trees, Random Forest, XGBoost) are scale-invariant and generally do NOT require scaling.

1.2 Key Techniques

A. Standardization (Z-score scaling)

Transforms data to have mean = 0 and standard deviation = 1. Best when data is roughly Gaussian, or for algorithms that assume normally distributed input.

Formula: $z = (x - \text{mean}) / \text{std}$

Standardization formula

B. Min-Max Normalization

Rescales values into a fixed range, usually [0, 1]. Sensitive to outliers since it uses min/max directly.

Formula: $x_{\text{scaled}} = (x - \text{min}) / (\text{max} - \text{min})$

Min-Max formula

C. Robust Scaling

Uses median and interquartile range (IQR) instead of mean/std, making it robust to outliers.

Formula: $x_{\text{scaled}} = (x - \text{median}) / \text{IQR}$

Robust scaling formula

D. MaxAbs Scaling & Unit Vector (L2) Normalization

- MaxAbs Scaling: divides each value by the maximum absolute value; preserves sparsity, useful for sparse matrices (e.g., TF-IDF).
- L2 Normalization (Normalizer): scales each ROW (sample) to have unit norm — useful for text/vector data, not per-column like the others.

1.3 Comparison Table

Method	Range/Output	Outlier Sensitive?	Typical Use Case
StandardScaler	Mean 0, Std 1	Yes	Linear/Logistic Regression, PCA, SVM, Neural Nets
MinMaxScaler	[0, 1] (custom range)	Very sensitive	Neural nets, image pixel data, bounded features
RobustScaler	Centered on median	Robust	Data with outliers
MaxAbsScaler	[-1, 1]	Somewhat	Sparse data (TF-IDF, one-hot)
Normalizer (L2)	Unit norm per row	N/A (row-wise)	Text vectors, cosine similarity tasks

1.4 Python Implementation

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler, MaxAbsScaler, Normalizer
)

# Sample dataset
df = pd.DataFrame({
    "age": [25, 32, 47, 51, 62, 23, 90],
    "income": [40000, 52000, 61000, 300000, 82000, 39000, 75000]
})

# ---- 1. Standardization (Z-score) ----
scaler = StandardScaler()
df["age_std"] = scaler.fit_transform(df[["age"]])

# ---- 2. Min-Max Normalization ----
minmax = MinMaxScaler(feature_range=(0, 1))
df["age_minmax"] = minmax.fit_transform(df[["age"]])

# ---- 3. Robust Scaling (handles outliers, e.g. income) ----
robust = RobustScaler()
df["income_robust"] = robust.fit_transform(df[["income"]])

# ---- 4. MaxAbs Scaling ----
maxabs = MaxAbsScaler()
df["income_maxabs"] = maxabs.fit_transform(df[["income"]])

print(df)
```

Example: applying multiple scalers with scikit-learn

```
# Always fit on TRAIN data only, then transform both train & test
# to avoid data leakage.

from sklearn.model_selection import train_test_split

X = df[["age", "income"]]
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)    # fit + transform  
X_test_scaled  = scaler.transform(X_test)        # transform ONLY
```

Correct train/test scaling workflow (avoiding data leakage)

2. Encoding Categorical Variables

2.1 Why It Matters

Machine learning models generally require numerical input. Categorical (text/label) features such as "color", "city", or "education level" must be converted into numeric form before modeling. The right encoding strategy depends on cardinality, whether the category is ordinal or nominal, and the model being used.

2.2 Key Techniques

A. Label Encoding

Assigns each category an integer (0, 1, 2, ...). Simple, but implies an ordinal relationship that may not exist — risky for nominal data with linear/distance-based models. Safe for tree-based models and truly ordinal features.

B. One-Hot Encoding

Creates a new binary column for each category. Avoids false ordinality but increases dimensionality — problematic for high-cardinality features ("curse of dimensionality").

C. Ordinal Encoding

Like label encoding, but the integer order is explicitly and meaningfully assigned (e.g., Low=0, Medium=1, High=2). Correct choice for genuinely ordered categories.

D. Frequency / Count Encoding

Replaces each category with its frequency (or count) in the dataset. Compact and works well with high-cardinality features, though it can collide categories with equal frequency.

E. Target (Mean) Encoding

Replaces each category with the mean of the target variable for that category. Powerful for high-cardinality features but prone to overfitting/target leakage — requires cross-validation or smoothing.

F. Binary / Hashing Encoding

Binary encoding converts category integers to binary digits split across columns ($\log_2(N)$ columns instead of N). Hashing encoding maps categories into a fixed number of buckets via a hash function — useful for very high-cardinality or streaming data.

2.3 Comparison Table

Method	Best For	Dimensionality	Risk
Label Encoding	Ordinal or tree-based models	Low (1 col)	False ordinal assumption
One-Hot Encoding	Low-cardinality, nominal	High (N cols)	Curse of dimensionality
Ordinal Encoding	True ordered categories	Low (1 col)	Wrong order = bad signal
Frequency Encoding	High-cardinality nominal	Low (1 col)	Category collisions
Target Encoding	High-cardinality, supervised	Low (1 col)	Overfitting / leakage
Hashing Encoding	Very high-cardinality / streaming	Fixed (k cols)	Hash collisions

2.4 Python Implementation

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, OneHotEncoder

df = pd.DataFrame({
    "city": ["Delhi", "Mumbai", "Delhi", "Chennai", "Mumbai"],
    "education": ["Bachelors", "Masters", "PhD", "Bachelors", "Masters"],
    "target": [1, 0, 1, 0, 1]
})

# ---- A. Label Encoding ----
le = LabelEncoder()
df["city_label"] = le.fit_transform(df["city"])

# ---- B. One-Hot Encoding (pandas) ----
df_ohe = pd.get_dummies(df, columns=["city"], prefix="city")

# ---- B (alt). One-Hot Encoding (sklearn, fits into pipelines) ----
ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
city_ohe = ohe.fit_transform(df[["city"]])

# ---- C. Ordinal Encoding (explicit order) ----
edu_order = ["Bachelors", "Masters", "PhD"]
ord_enc = OrdinalEncoder(categories=edu_order)
df["education_ordinal"] = ord_enc.fit_transform(df[["education"]])

print(df)
print(df_ohe)
```

Label, One-Hot, and Ordinal encoding with pandas / scikit-learn

```
# ---- D. Frequency Encoding ----
freq_map = df["city"].value_counts(normalize=True)
df["city_freq"] = df["city"].map(freq_map)

# ---- E. Target (Mean) Encoding – with K-Fold to avoid leakage ----
from sklearn.model_selection import KFold
import numpy as np

df["city_target_enc"] = np.nan
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```

for train_idx, val_idx in kf.split(df):
    train_fold, val_fold = df.iloc[train_idx], df.iloc[val_idx]
    means = train_fold.groupby("city")["target"].mean()
    df.loc[val_idx, "city_target_enc"] = df.loc[val_idx, "city"].map(means)

# fill any unseen categories with the global mean
df["city_target_enc"].fillna(df["target"].mean(), inplace=True)

print(df[["city", "city_freq", "city_target_enc"]])

```

Frequency encoding and leakage-safe K-Fold target encoding

```

# ---- F. Category-Encoders library (Binary / Hashing / Target) ----
# pip install category_encoders
import category_encoders as ce

binary_enc = ce.BinaryEncoder(cols=["city"])
df_binary = binary_enc.fit_transform(df["city"])

hash_enc = ce.HashingEncoder(cols=["city"], n_components=4)
df_hashed = hash_enc.fit_transform(df["city"])

target_enc = ce.TargetEncoder(cols=["city"])
df["city_target_ce"] = target_enc.fit_transform(df["city"], df["target"])

```

Binary, hashing, and target encoding using the category_encoders library

3. Handling Skewness

3.1 Why It Matters

Skewness measures the asymmetry of a distribution. Many statistical models (linear regression, logistic regression, LDA) assume features are approximately normally distributed. Highly skewed features can violate this assumption, compress the influence of the majority of data points, and make models overly sensitive to extreme values.

- Right (positive) skew: long tail on the right — common in income, prices, population counts.
- Left (negative) skew: long tail on the left — less common, e.g., age at retirement.
- Rule of thumb:
 - Skewness ≈ 0 → approximately symmetric
 - 0 to +1 → moderate positive skew
 - $> +1$ → strong positive skew
 - -1 to 0 → moderate negative skew
 - < -1 → strong negative skew

Symmetric distribution

```
*
***
*****
*****
Mean  $\approx$  Median
```

Right-skewed distribution

```
**
***
*****
*****
*****
*****
*
```

The distribution has a long right tail.

Examples:

- Salary
- House prices
- Income

- Transaction amount

3.2 Detecting Skewness

```
import pandas as pd
from scipy.stats import skew

df = pd.DataFrame({"income": [22000, 25000, 27000, 30000, 32000,
                              35000, 40000, 250000, 300000, 500000]})

print("Skewness:", skew(df["income"]))          # scipy method
print("Pandas skew:", df["income"].skew())      # pandas built-in

# Visual check
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df[income], kde=True)
plt.title("Income Distribution")
plt.show()
```

Measuring skewness numerically and visually

3.3 Key Transformation Techniques

A. Log Transformation

Compresses large values, effective for right-skewed data with all-positive values. Use $\log_{1p}(\log(1+x))$ to safely handle zeros.

B. Square Root Transformation

A gentler transform than log; works for moderately right-skewed, non-negative data (e.g., count data).

C. Box-Cox Transformation

A parametrized family of power transforms that finds the optimal λ to best normalize the data. Requires strictly positive values.

D. Yeo-Johnson Transformation

A generalization of Box-Cox that works with zero and negative values as well — the more flexible, general-purpose choice.

E. Winsorization / Capping

Caps extreme values at a chosen percentile (e.g., 1st and 99th) rather than transforming the whole distribution — reduces the influence of outliers driving skew.

3.4 Comparison Table

Method	Handles Zero/Negative?	Strength
Log (log1p)	Zero OK, not negative	Strong compression, simple, interpretable
Square Root	Zero OK, not negative	Mild compression
Box-Cox	No (must be > 0)	Optimal λ , strong normalization
Yeo-Johnson	Yes	Most flexible, works on any real value
Winsorization	Yes	Targets outliers specifically, not full shape

3.5 Python Implementation

```
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.preprocessing import PowerTransformer

df = pd.DataFrame({"income": [22000, 25000, 27000, 30000, 32000,
                             35000, 40000, 250000, 300000, 500000]})

# ---- A. Log Transform ----
df["income_log"] = np.log1p(df["income"])          # log(1 + x), safe for zeros

# ---- B. Square Root Transform ----
df["income_sqrt"] = np.sqrt(df["income"])

# ---- C. Box-Cox (requires strictly positive values) ----
df["income_boxcox"], best_lambda = stats.boxcox(df["income"])
print("Optimal Box-Cox lambda:", best_lambda)

# ---- D. Yeo-Johnson (handles zero/negative) ----
pt = PowerTransformer(method="yeo-johnson")
df["income_yeojohnson"] = pt.fit_transform(df[["income"]])

# ---- E. Winsorization (cap at 5th / 95th percentile) ----
lower, upper = df["income"].quantile([0.05, 0.95])
df["income_winsorized"] = df["income"].clip(lower=lower, upper=upper)

print(df)
print("Skew before:", df["income"].skew())
print("Skew after log:", df["income_log"].skew())
```

Applying and comparing skewness-reduction transforms

Tip: Always compare skewness before/after transformation, and remember to inverse-transform predictions back to the original scale if the target variable itself was transformed.

4. Feature Selection Methods

4.1 Why It Matters

Feature selection reduces the number of input variables to those that are most relevant, improving model performance, reducing overfitting, cutting training time, and improving interpretability.

Feature Selection vs Feature Extraction

Feature Selection

Select existing features.

A B C D E

↓

A C E

Feature Extraction

Create new features from existing ones.

Example:

A + B + C

↓

New Feature

PCA is a feature extraction technique rather than ordinary feature selection.

Feature Selection Methods are commonly grouped into three families: Filter, Wrapper, and Embedded methods.

4.2 Filter Methods

Select features using statistical measures, independent of any ML model. Fast and scalable, but ignore feature interactions.

- Variance Threshold: removes features with little/no variance.
- Correlation analysis: drops features highly correlated with each other (multicollinearity) or weakly correlated with the target.
- Chi-Square Test: measures dependence between categorical features and a categorical target.
- ANOVA F-test / mutual information: measures relationship strength between numeric features and target.

4.3 Wrapper Methods

Use a predictive model to evaluate subsets of features, searching for the combination that yields the best performance. More accurate than filter methods but computationally expensive.

- Recursive Feature Elimination (RFE): repeatedly builds a model, ranks features by importance, and removes the weakest.
- Forward Selection: starts empty, adds the most useful feature one at a time.
- Backward Elimination: starts with all features, removes the least useful one at a time.

4.4 Embedded Methods

Feature selection happens as part of model training itself — combining the speed of filter methods with the model-awareness of wrapper methods.

- Lasso (L1) Regression: shrinks less important feature coefficients to exactly zero.
- Tree-based feature importance: Random Forest / Gradient Boosting expose `feature_importances_` after training.
- Elastic Net: combines L1 and L2 penalties for correlated feature groups.

4.5 Comparison Table

Category	Examples	Speed	Model-Aware?
Filter	Variance Threshold, Chi-Square, Correlation, ANOVA	Fast	No
Wrapper	RFE, Forward/Backward Selection	Slow (many model fits)	Yes
Embedded	Lasso, Random Forest Importance, Elastic Net	Moderate	Yes (built into training)

4.6 Python Implementation

Filter Method: Variance Threshold & Correlation

```
import pandas as pd
import numpy as np
from sklearn.feature_selection import VarianceThreshold

df = pd.DataFrame({
```

```

    "f1": [1, 1, 1, 1, 1],          # zero variance -> useless
    "f2": [1, 2, 3, 4, 5],
    "f3": [10, 9, 8, 7, 6],
    "target": [0, 1, 0, 1, 1]
})

# ---- Variance Threshold ----
selector = VarianceThreshold(threshold=0.0)
selector.fit(df[["f1", "f2", "f3"]])
kept_cols = df[["f1", "f2", "f3"]].columns[selector.get_support()]
print("Kept after variance threshold:", list(kept_cols))

# ---- Correlation with target ----
corr = df.corr()["target"].drop("target").abs().sort_values(ascending=False)
print("Correlation with target:\n", corr)

```

Filter-based selection using variance and correlation

Filter Method: Chi-Square & ANOVA F-test

```

from sklearn.feature_selection import SelectKBest, chi2, f_classif
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

# ---- Chi-Square (requires non-negative features) ----
chi_selector = SelectKBest(score_func=chi2, k=2)
X_chi = chi_selector.fit_transform(X, y)
print("Chi2 scores:", chi_selector.scores_)

# ---- ANOVA F-test (numeric features, categorical target) ----
f_selector = SelectKBest(score_func=f_classif, k=2)
X_f = f_selector.fit_transform(X, y)
print("ANOVA F-scores:", f_selector.scores_)

```

Univariate statistical feature selection with SelectKBest

Wrapper Method: Recursive Feature Elimination (RFE)

```

from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)

model = LogisticRegression(max_iter=1000)
rfe = RFE(estimator=model, n_features_to_select=2)
rfe.fit(X, y)

print("Selected features (mask):", rfe.support_)
print("Feature ranking:", rfe.ranking_)  # 1 = selected

```

Wrapper-based selection using Recursive Feature Elimination

Embedded Method: Lasso & Random Forest Importance

```

from sklearn.linear_model import Lasso
from sklearn.ensemble import RandomForestClassifier

```

```

import pandas as pd

X, y = load_iris(return_X_y=True)
feature_names = [f"feature_{i}" for i in range(X.shape[1])]

# ---- Lasso (L1) – coefficients shrink to zero for weak features ----
lasso = Lasso(alpha=0.05)
lasso.fit(X, y)
lasso_importance = pd.Series(lasso.coef_, index=feature_names)
print("Lasso coefficients:\n", lasso_importance)

# ---- Random Forest feature importance ----
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X, y)
rf_importance = pd.Series(rf.feature_importances_, index=feature_names)
print("Random Forest importances:\n", rf_importance.sort_values(ascending=False))

```

Embedded feature selection using Lasso regularization and tree-based importance

Tip: Combine methods in practice: use a fast filter step to remove obviously irrelevant features, then a wrapper/